PIPELINES DE APRENDIZADO DE MÁQUINA

EAB067 - Aprendizado de Máquina

Atualizado em: 9 de março de 2026

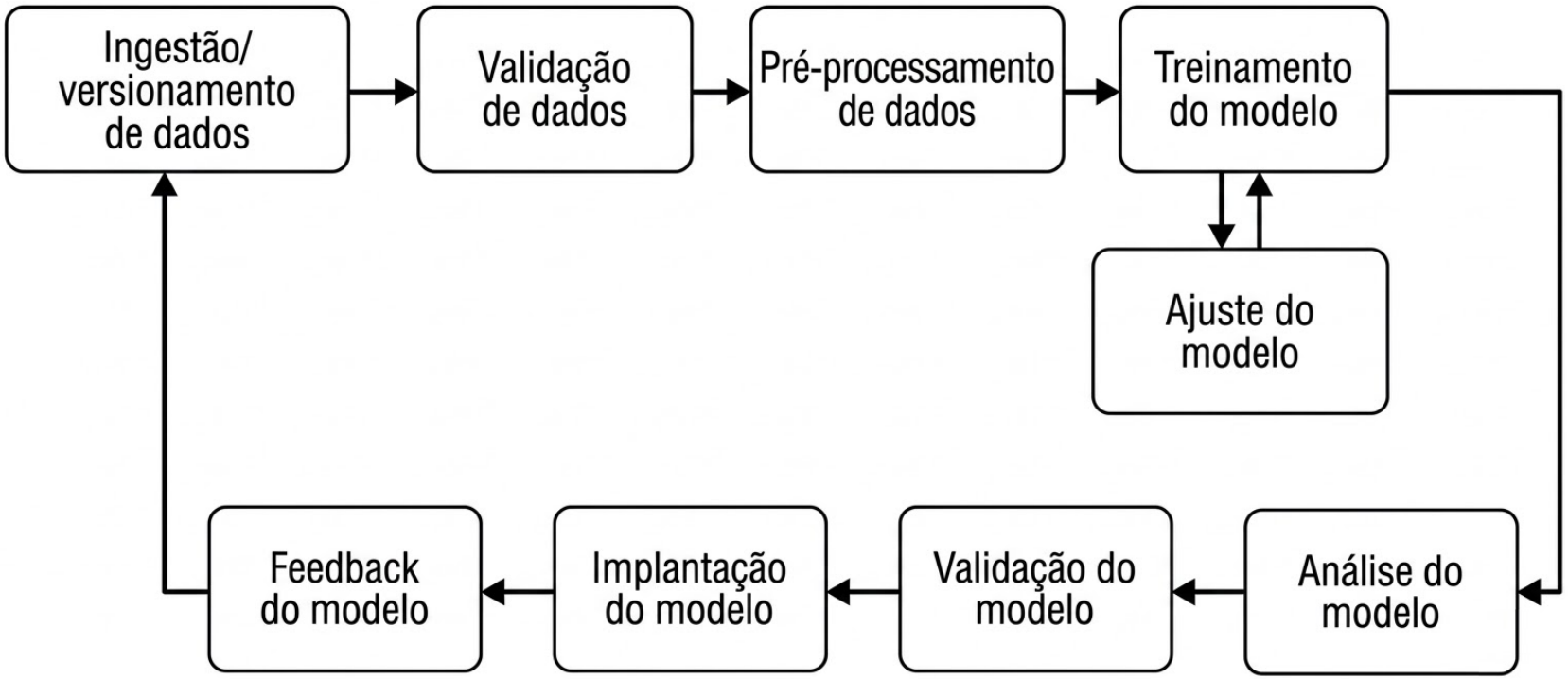
Iago Carvalho

Programa de Pós-Graduação em
Estatística Aplicada e Biometria

PIPELINES DE APRENDIZADO DE MÁQUINA

Um pipeline é uma série de estágios interconectados que automatizam o fluxo de trabalho completo

- Ingestão e tratamento de dados
- Definição, treino, teste e avaliação de modelos
- Implantação e integração contínua (CI/CD)



PORQUÊ MONTAR UM PIPELINE DE APRENDIZADO?

Pipelines de aprendizado de máquina **automatizam**, **simplificam** e **padronizam** todo o fluxo de trabalho

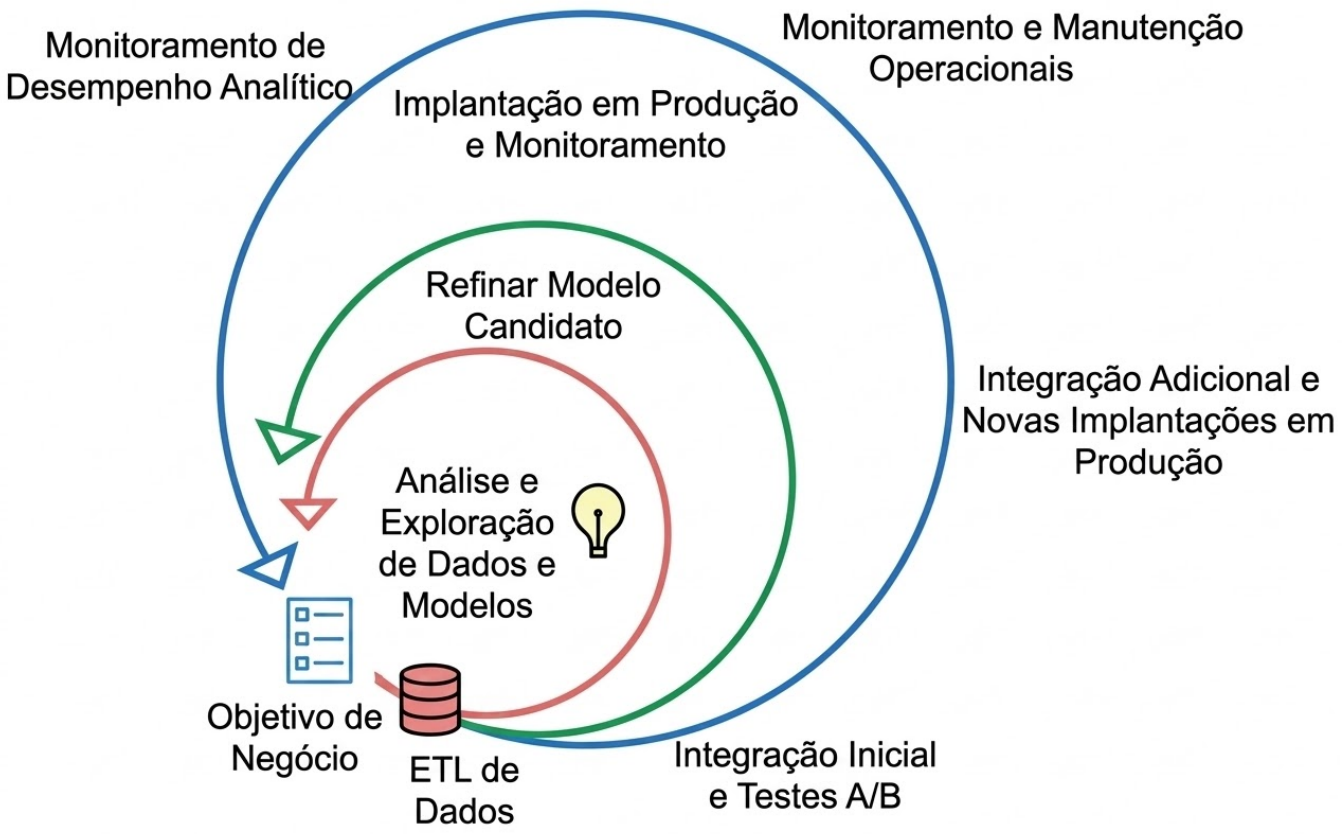
Podem **reduzir erros** humanos, garantir a **reprodutibilidade**, viabilizar a **escalabilidade** e acelerar a **transição** da pesquisa experimental para aplicações prontas para produção.

Podemos elencar cinco grandes vantagens de utilizar pipelines

1. Automação e eficiência
2. Reprodutibilidade e depuração
3. Escalabilidade
4. Colaboração aprimorada
5. Rápida implantação (deploy)
6. Integridade e monitoramento de dados

LOOPS DE DESENVOLVIMENTO

Podemos dividir um pipeline em três tarefas iterativas



LOOP INTERNO

O loop interno trata de exploração e experimentação

- Normalmente, ele tem diversas iterações rápidas
- O objetivo é testar e avaliar diversas abordagens para ver qual funciona melhor

Nesta fase, todos os modelos são *offline*

- Evita o custo de implantação

Deve-se obter *insights* iniciais sobre o que funciona melhor

- Qual é o melhor modelo
- Quais técnicas de pré-processamento são necessárias e/ou efetivas
- Quais são as variáveis (features) mais importantes

LOOP INTERMEDIÁRIO

As técnicas. modelos e algoritmos do *loop* anterior são aprimorados

- Ajuste dos hiperparâmetros dos modelos
- Reavaliação das métricas de qualidade

Nesta etapa, o objetivo é reduzir a quantidade de modelos candidatos obtidos no *loop* anterior

- Somente um subconjunto dos modelos iniciais vai prosseguir para o *loop* externo

Em geral, aqui todos os esforços ainda estão *offline*

LOOP EXTERNO

Aqui é onde as coisas *ficam de verdade* e os modelos refinados são implantados em ambiente de produção

- Modelos integrados com sistemas existentes
- Incorporação de fluxo de dados para monitoramento e treinamento dos modelos desenvolvidos

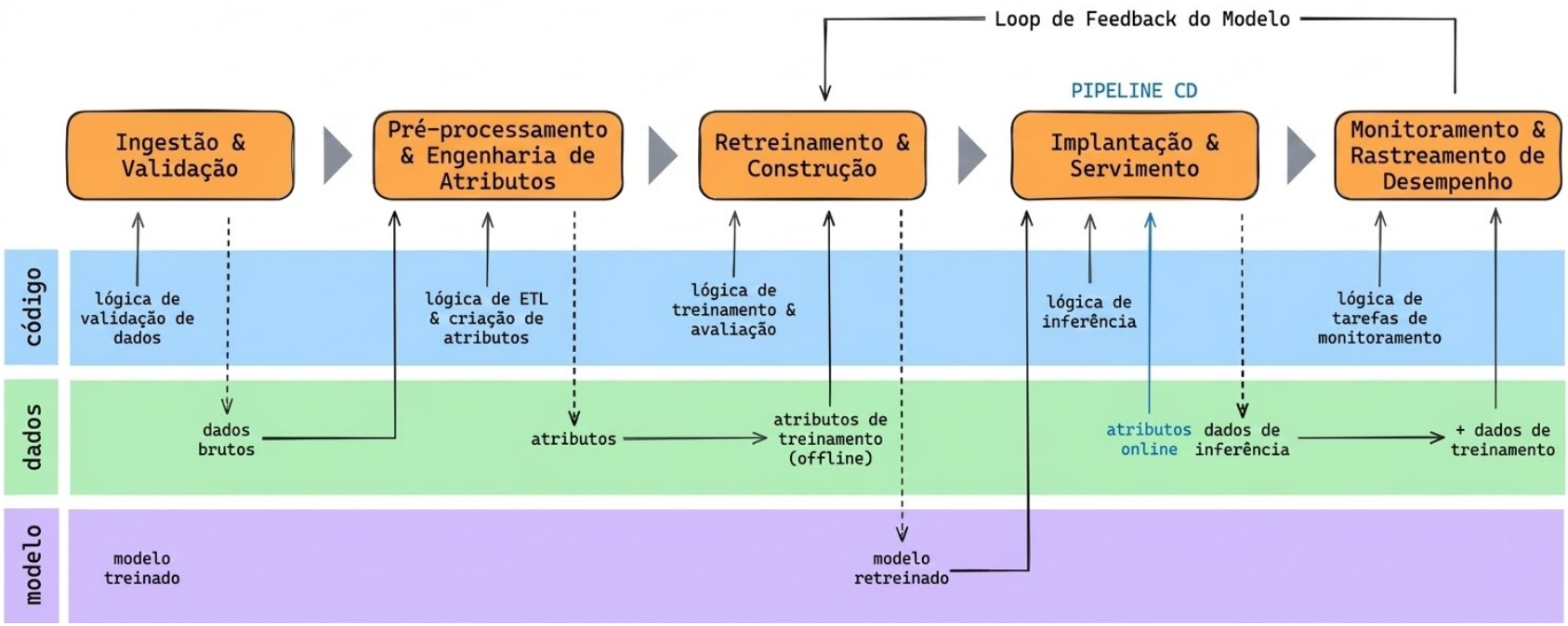
Nesta etapa, o modelo já está pronto. Agora, a principal preocupação é com sua implantação (MLOps)

- Modelos desenvolvidos tem que ser confiáveis, escaláveis e eficientes em ambientes reais

O objetivo principal não é obter os modelos em si, mas configurar toda infraestrutura necessária para sua execução, monitoramento e avaliação contínua

EXEMPLO DE PIPELINE

Vamos estudar o seguinte pipeline de ML dividido em 5 fases



INGESTÃO E VALIDAÇÃO

Consiste em coletar dados de diversas fontes e garantir sua usabilidade

- Dados limpos e estruturados
- Algumas transformações podem ser necessárias

Esta etapa pode ser entendida como um pré-processamento dos dados

- É aqui que vamos garantir que os dados estejam livres de anomalias ou valores ausentes que possam distorcer o processo de aprendizado

INGESTÃO E VALIDAÇÃO - FONTES DE DADOS

Ter diversas fontes de dados pode ser uma dávida ou uma maldição

- Modelos de ML gostam de muitos dados
- Podem haver dados duplicados
- Provavelmente será necessário um tratamento diferente para cada fonte de dados

Validação de dados pode ser realizado de forma manual ou com ferramentas adicionais

- Proprietárias: Google Dataflow, Amazon Glue, Amazon Kinesis
- Abertas: Spark (*batches*) e Kafka (tempo real)

INGESTÃO E VALIDAÇÃO - QUALIDADE DOS DADOS

Um bom banco de dados normalmente não possui dados faltantes ou values nulos

- Ele também não possui dados duplicados

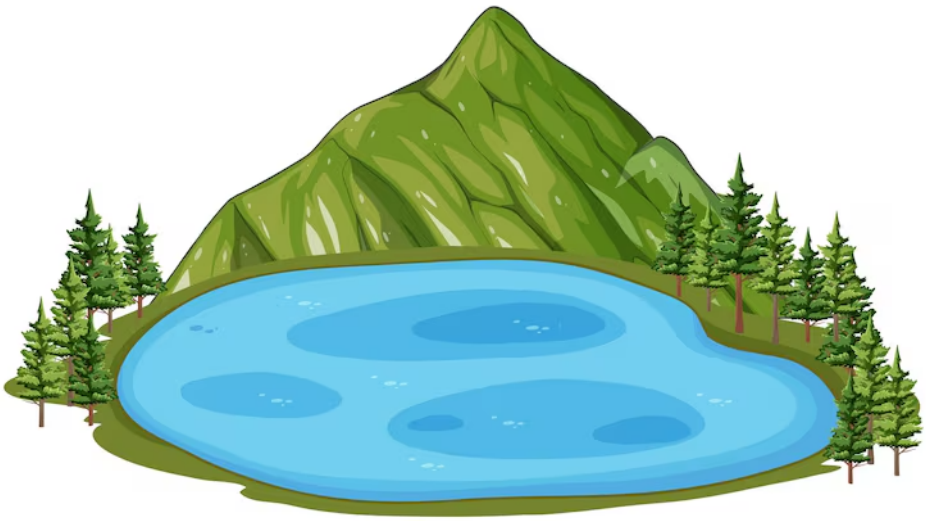
Além disso, também deve-se verificar o *range* dos dados e a consistência de seus tipos

Uma ótima ferramenta para fazer a análise exploratória de dados é o ydata-profiling

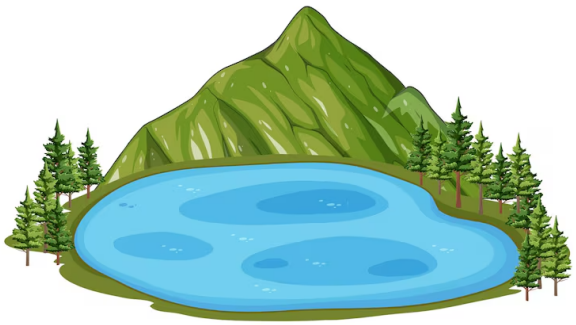
INGESTÃO E VALIDAÇÃO - BANCOS DE DADOS

Tão importante quanto tratar os dados é salva-los de forma estruturada

- Arquivos .csv ou .json, bancos de dados SQL e NoSQL
- Serviços de nuvem como Amazon S3 e Google Cloud Storage



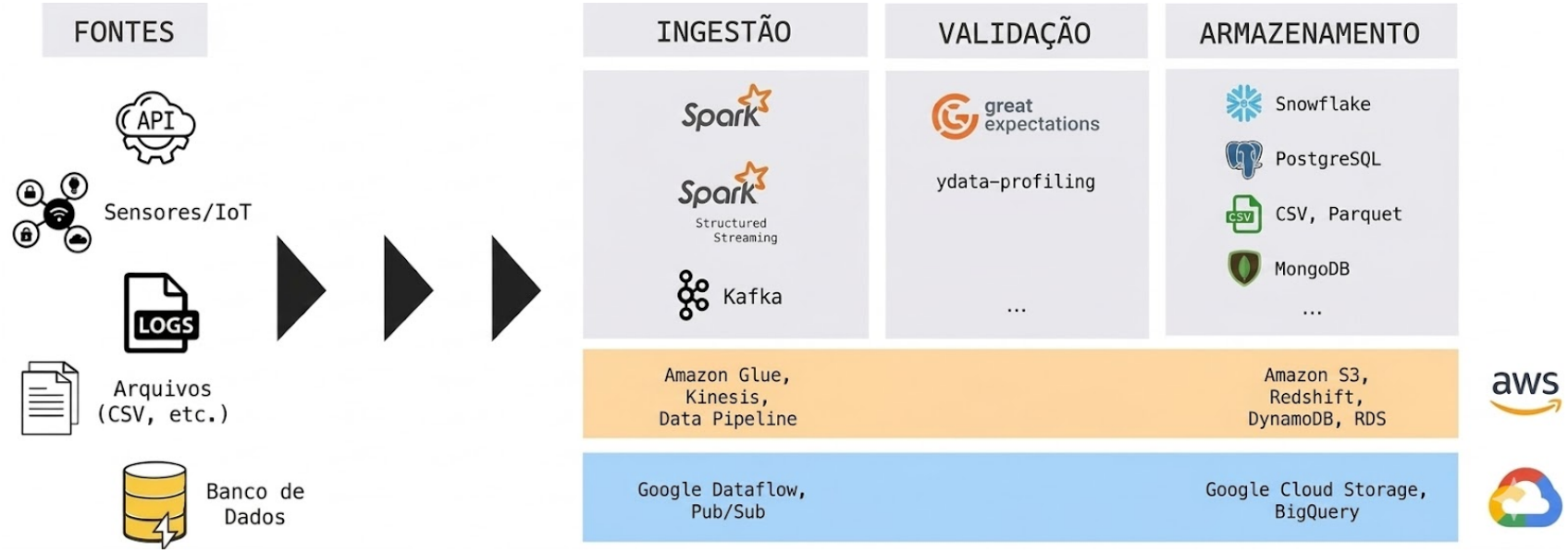
INGESTÃO E VALIDAÇÃO - BANCOS DE DADOS



- Dados não estruturados, semi-estruturados e estruturados
- Construído utilizando ELT
- Dados usados por engenheiros, desenvolvedores e cientistas
- Utilizado para construir modelos de ML, análises preditivas e exploratórias

- Dados estruturados e processados
- Construído utilizando ETL
- Dados utilizados por gerentes e analistas
- Utilizado para criar *dashboards* e relatórios de negócio

INGESTÃO E VALIDAÇÃO - FERRAMENTAS



PRÉ-PROCESSAMENTO E ENGENHARIA DE ATRIBUTOS

O objetivo aqui é fazer a "otimização"dos dados

- Preparar os dados para que o modelo aprenda da melhor forma possível

Esta fase inclui reaizar a normalização e padronização dos dados, codificar variáveis categórias e utilizar estratégias avançadas

- *Feature extraction* (PCA, LDA, SVD, ...)
- *Feature selection* (métodos *wrapper*, análise de variância, regularização L1, etc)
- Balanceamento do banco de dados (SMOTE, ADASYN, undersampling, ...)
- Imputação de dados faltantes (média, mediana, *carry on*, RF, LLM, ...)

PRÉ-PROCESSAMENTO

Normalmente, dados para algoritmos de ML vem de um *data lake*

- Dados sem nenhum pré-processamento

Deste modo, devemos fazer a normalização dos dados para garantir que todos tenham a mesma importância

- Por exemplo, colocar todos os dados numéricos no intervalo $[0, 1]$

Variáveis categóricas devem ser codificadas

Além disso, *outliers* podem ser removidos (ou não)

FEATURE EXTRACTION E *FEATURE SELECTION*

Feature extraction trata de criar novas variáveis a partir das variáveis existentes

- Criar novas variáveis como combinações (lineares) das existentes
- Extrair o campo *hora* de um *timestamp*
- Criar novas variáveis resumindo séries temporais (e.g., médias móveis)
- ...

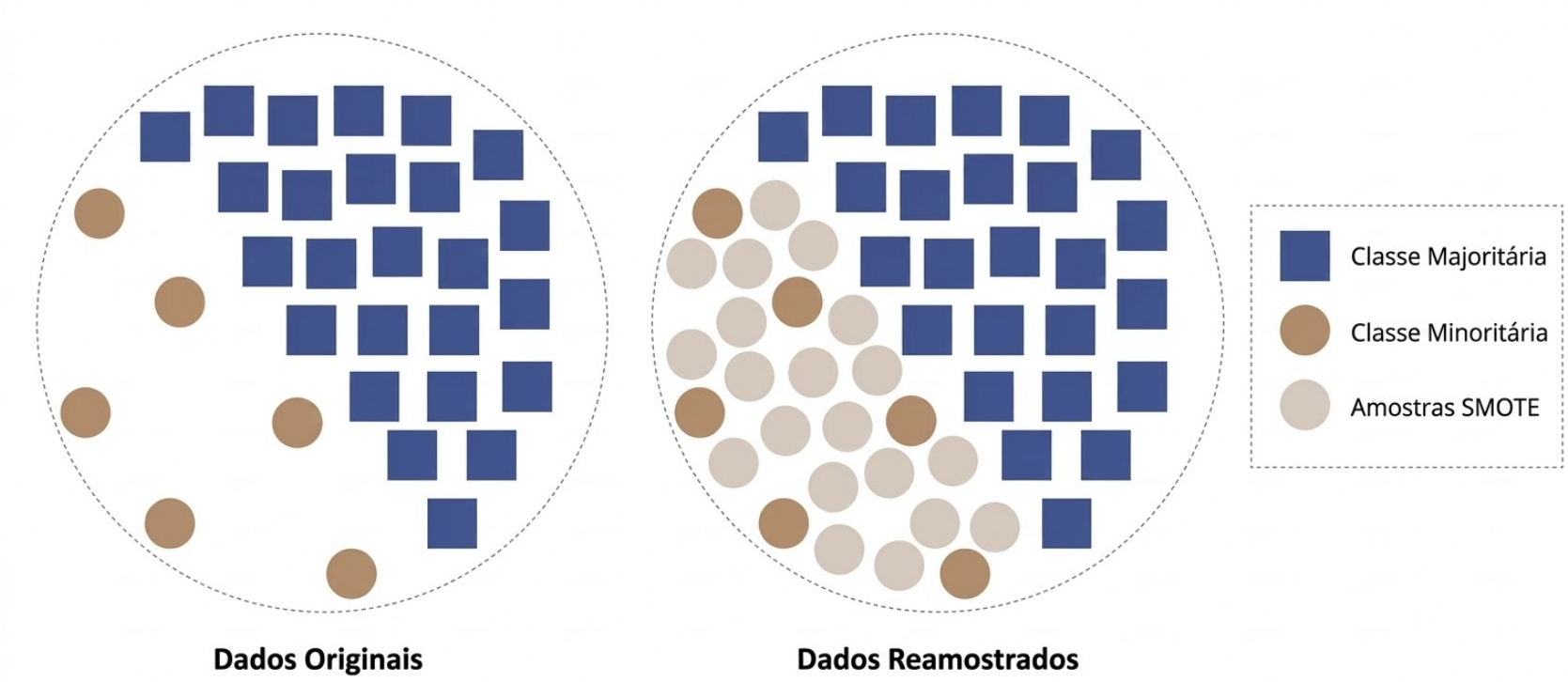
Já *Feature selection* é a tarefa de selecionar as variáveis mais importantes do modelo

- Retirar variáveis altamente correlacionadas
- Reduzir o ruído residual no modelo de aprendizado
- Diminuir o custo computacional do algoritmo de ML

BALANCEAMENTO DO BANCO DE DADOS

Em tarefas de classificação, é muito comum termos desbalanceamento de classes

- Muitas amostras de uma classe, poucas amostras de outras
- O algoritmo é incapaz de generalizar a classe com poucas amostras
- É necessário balancear o número de amostras



IMPUTAÇÃO DE DADOS

As vezes, algum dado pode ser faltante por diversos motivos

- Modelos de ML normalmente não conseguem tratar este dado faltante
- Deve-se, de alguma forma, estimar o dado faltante e completar o banco de dados

	col1	col2	col3	col4	col5
0	2	5.0	3.0	6	NaN
1	9	NaN	9.0	0	7.0
2	19	17.0	NaN	9	NaN

mean() →

	col1	col2	col3	col4	col5
0	2.0	5.0	3.0	6.0	7.0
1	9.0	11.0	9.0	0.0	7.0
2	19.0	17.0	6.0	9.0	7.0

CONSTRUÇÃO E RETREINAMENTO DE MODELOS

Um bom modelo de ML é um modelo bem treinado

- Divisão dos dados em treinamento e teste
- Hiperparâmetros otimizados

Entretanto, um modelo de ML não é estático

- Dados novos chegam a todo momento
- Modelos tem que ser constantemente atualizados

Junto ao retreinamento, vem também a necessidade de criar o versionamento dos modelos

- Talvez seja necessário reverter o modelo para um estado anterior de melhor qualidade

CONSTRUÇÃO E RETREINAMENTO DE MODELOS

Um bom modelo de ML é um modelo bem treinado

- Divisão dos dados em treinamento e teste
- Hiperparâmetros otimizados

Entretanto, um modelo de ML não é estático

- Dados novos chegam a todo momento
- Modelos tem que ser constantemente atualizados

Junto ao retreinamento, vem também a necessidade de criar o versionamento dos modelos

- Talvez seja necessário reverter o modelo para um estado anterior de melhor qualidade

OTIMIZAÇÃO DE HIPERPARÂMETROS

Basicamente, todos os modelos de ML possuem um ou mais hiperparâmetros

- Número e variedade de algoritmos em um *ensemble*
- Quantidade de árvores em um algoritmo Random Forest ou XGBoost

O ajuste dos valores destes hiperparâmetros pode melhorar (e muito!) o desempenho de um modelo de ML

Existem diversas técnicas para ajuste de hiperparâmetros

- Busca aleatória, *grid search*, Optuna

DIVISÃO DE DADOS EM TREINAMENTO E TESTE

Como o próprio nome diz, dados de treinamento são utilizados para treinar o modelo

- Dados de teste são utilizados para testar e avaliar o modelo

Existem diversas abordagens para realizar o treinamento e teste

- Leave One Out Cross-validation
- K-Fold Cross-validation
- Forward chaining (para séries temporais)
- Monte Carlos Cross-validation

Um modelo bem treinado é um modelo com bons resultados!

RETREINAMENTO

Em produção, modelos de ML não são estáticos

- Dados novos chegam a todo momento

É necessário realizar o retreinamento do modelo

- Retreinamento periódico
- Retreinamento por performance
- Retreinamento contínuo

Necessário uma arquitetura de software diferente

IMPLANTAÇÃO (DEPLOY) DE MODELOS

Na implantação de um modelo de ML você realmente vai precisar de uma arquitetura de software diferente

- Não é mais possível utilizar *notebooks* como Jupyter, Google Colab ou scripts R

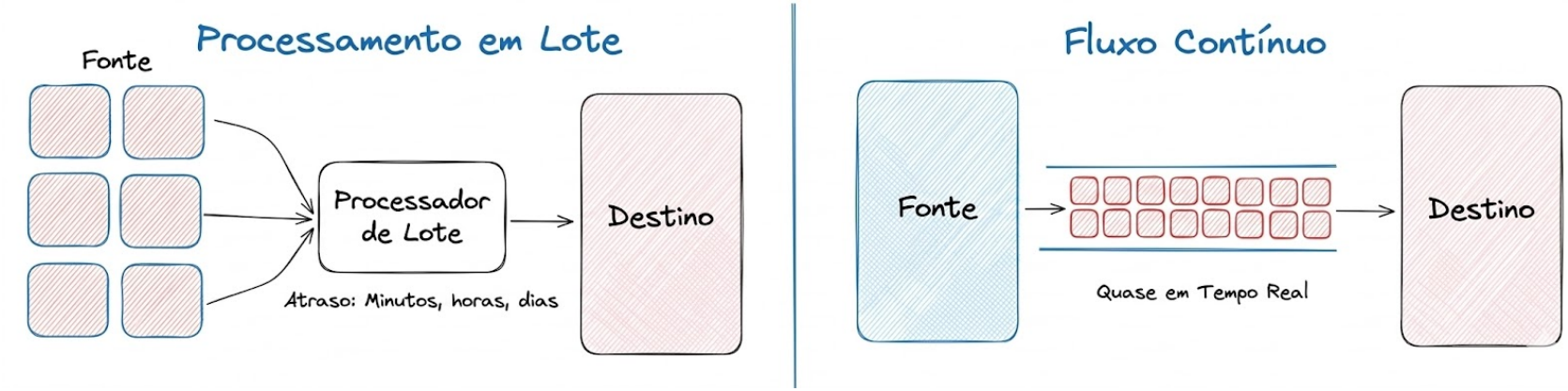
Necessário saber a finalidade do modelo para escolher a melhor maneira de implantação. Assim, temos três diferentes tipos de implantação

- Processamento em lotes (*batch*)
- Fluxo contínuo (*streaming*)
- Tempo real

IMPLANTAÇÃO (DEPLOY) DE MODELOS

Independentemente da maneira de implantação, devemos observar alguns requisitos

- Necessidade de testes A/B
- *Rollback* para modelos anteriores
  - Extremamente importante ter o versionamento dos modelos



MONITORAMENTO DE DESEMPENHO

Existem diversas formas de se avaliar o desempenho de um modelo

- Este monitoramento depende da estratégia de implantação utilizada
- Se o desempenho estiver baixo, é necessário retreinamento
  - Ou um ajuste na arquitetura do código
  - Ou um ajuste na infraestrutura computacional

Podemos dividir em dois três sub-tópicos

1. Monitoramento dos dados
2. Monitoramento de desempenho do modelo
3. Monitoramento da infraestrutura computacional

MONITORAMENTO DOS DADOS

Em modelos de fluxo contínuo ou de tempo real é necessário sempre checar e validar novos dados

- Garantir que não existem dados faltantes
- Remover (ou não) os *outliers*
- Verificar mudanças na distribuição das variáveis
- Monitorar por novos dados categóricos

Dados devem ser monitorados e ajustados antes de serem utilizados por modelos de ML

- Isto garante um bom desempenho do modelo durante todo seu ciclo de vida

MONITORAMENTO DE DESEMPENHO DO MODELO

Métricas de desempenho como coeficiente de determinação, acurácia ou *recall* são de extrema importância

Uma queda no desempenho de modelos pode indicar a necessidade de retreinamento

- Muito útil para modelos implantados com processamento em lote ou fluxo contínuo

Esta ideia pode ser adaptada também para realizar o *rollback* de modelos de tempo real

MONITORAMENTO DA INFRAESTRUTURA COMPUTACIONAL

Muito importante em modelos de tempo real ou de fluxo contínuo

- Estes podem tratar uma quantidade variável de dados ou servir a um número indefinido de usuários

Quanto mais dados e mais usuários, maior poder computacional é necessário

- Mais processamento, mais RAM, mais armazenamento, mais *endpoints* para comunicação

Este monitoramento tem que ser contínuo para garantir duas coisas

1. O modelo não vai "travar"
2. Evitar o desperdício de dinheiro

AUTOML

AutoML (Automated Machine Learning) tornam algoritmos de ML acessíveis a não especialistas, acelera projetos para cientistas de dados e melhora a eficiência na criação de modelos precisos

- Permite que pessoas sem conhecimento profundo em programação criem modelos de ML, economizando tempo e recursos
- Automatiza desde a preparação de dados brutos, seleção de variáveis até a escolha do melhor algoritmo
- Ajusta automaticamente hiperparâmetros, treino e teste de modelos
- São utilizados para facilitar a implementação de modelos de aprendizado em diversos setores da indústria

Existe uma certa "competição"para se otimizar pipelines AutoML